AI ASSISTED CODING

ASSIGNMENT-8.5

NAME:DANCHANALA PAVAN

ROLL NO:2403A52L07

BATCH:50

Task Description #1 (Username Validator – Apply AI in Authentication Context)

• Task: Use AI to generate at least 3 assert test cases for a function is_valid_username(username) and then implement the function using Test-Driven Development principles.

• Requirements:

o Username length must be between 5 and 15 characters.

o Must contain only alphabets and digits.

o Must not start with a digit.

o No spaces allowed.

Example Assert Test Cases:

```
assert is_valid_username("User123") == True
assert is_valid_username("12User") == False
assert is_valid_username("Us er") == False
```

Expected Output #1:

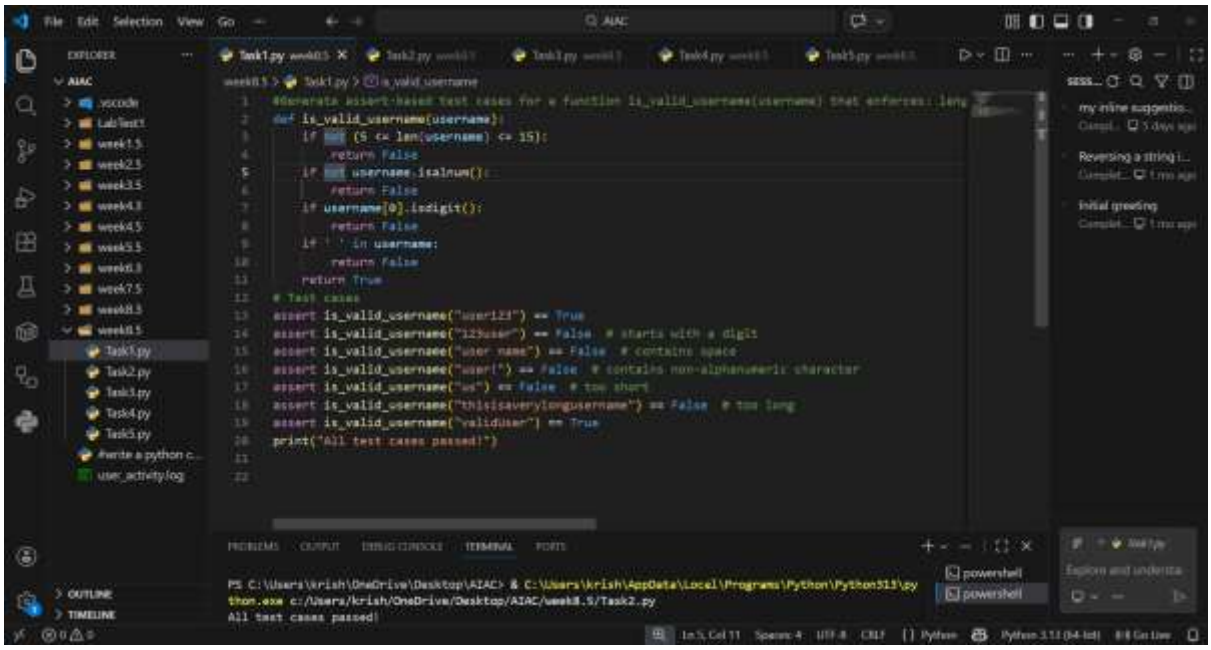
• Username validation logic successfully passing all AI-

generated test cases.

PROMPT:

“Generate assert-based test cases for a function is_valid_username(username) that enforces: length between 5–15 characters, only alphanumeric characters, must not start with a digit, and no spaces allowed”

CODE AND OUTPUT:



EXPLANATION:

We first define assert tests covering valid/invalid usernames. The function checks length (5–15), alphanumeric only, no spaces, and first character not a digit. Regex or string methods can be used. TDD ensures code passes all tests.

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

- **Task: Use AI to generate at least 3 assert test cases for a function classify_value(x) and implement it using conditional logic and loops.**
- **Requirements:**

o If input is an integer, classify as "Even" or "Odd".

o If input is 0, return "Zero".

o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

assert classify_value(8) == "Even"

assert classify_value(7) == "Odd"

assert classify_value("abc") == "Invalid Input"

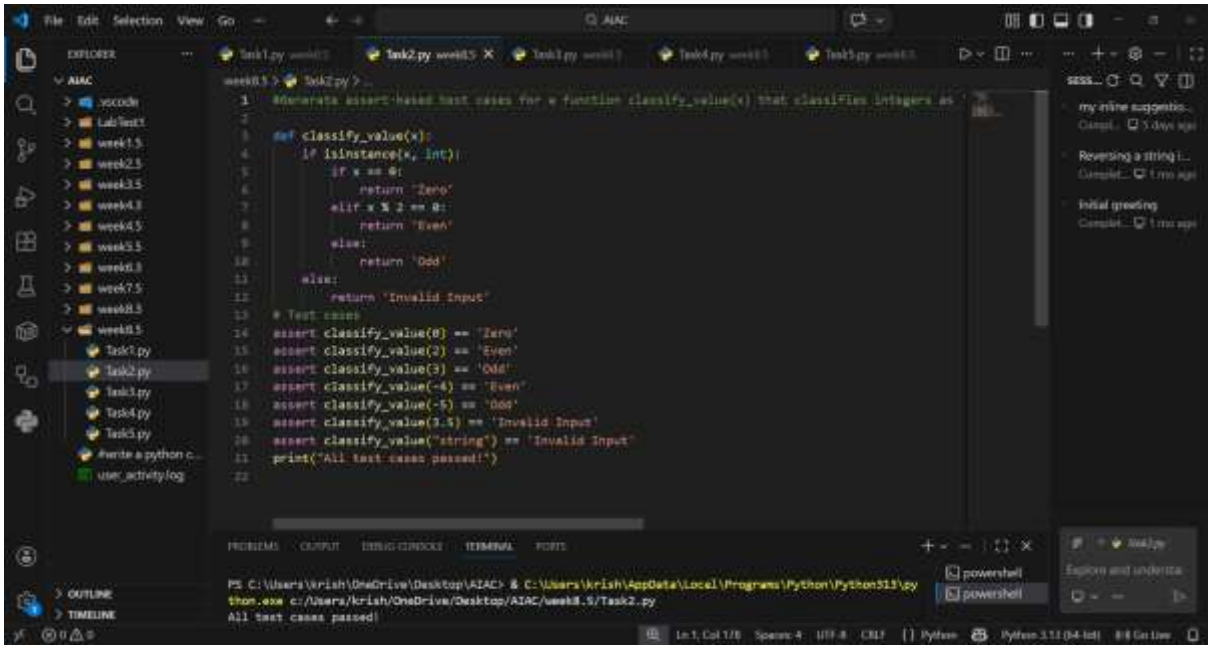
Expected Output #2:

• Function correctly classifying values and passing all test cases.

PROMPT:

“Generate assert-based test cases for a function classify_value(x) that classifies integers as 'Even' or 'Odd', returns 'Zero' for 0, and 'Invalid Input' for non-numeric values."

CODE AND OUTPUT:



EXPLANATION:

Tests include integers, zero, non-integers, and strings. The function uses type checking, then modulo for even/odd. Zero is handled separately. Non-numeric inputs return "Invalid Input". TDD ensures robustness.

def classify_value(x): if isinstance(x, int): if x == 0: return "Zero" return "Even" if x % 2 == 0 else "Odd" return "Invalid Input"

Task Description #3 (Palindrome Checker – Apply AI for

String Normalization)

• Task: Use AI to generate at least 3 assert test cases for a

function is_palindrome(text) and implement the function.

• Requirements:

o Ignore case, spaces, and punctuation.

o Handle edge cases such as empty strings and single

characters.

Example Assert Test Cases:

assert is_palindrome("Madam") == True

assert is_palindrome("A man a plan a canal Panama") ==

True

assert is_palindrome("Python") == False

Expected Output #3:

• Function correctly identifying palindromes and passing all

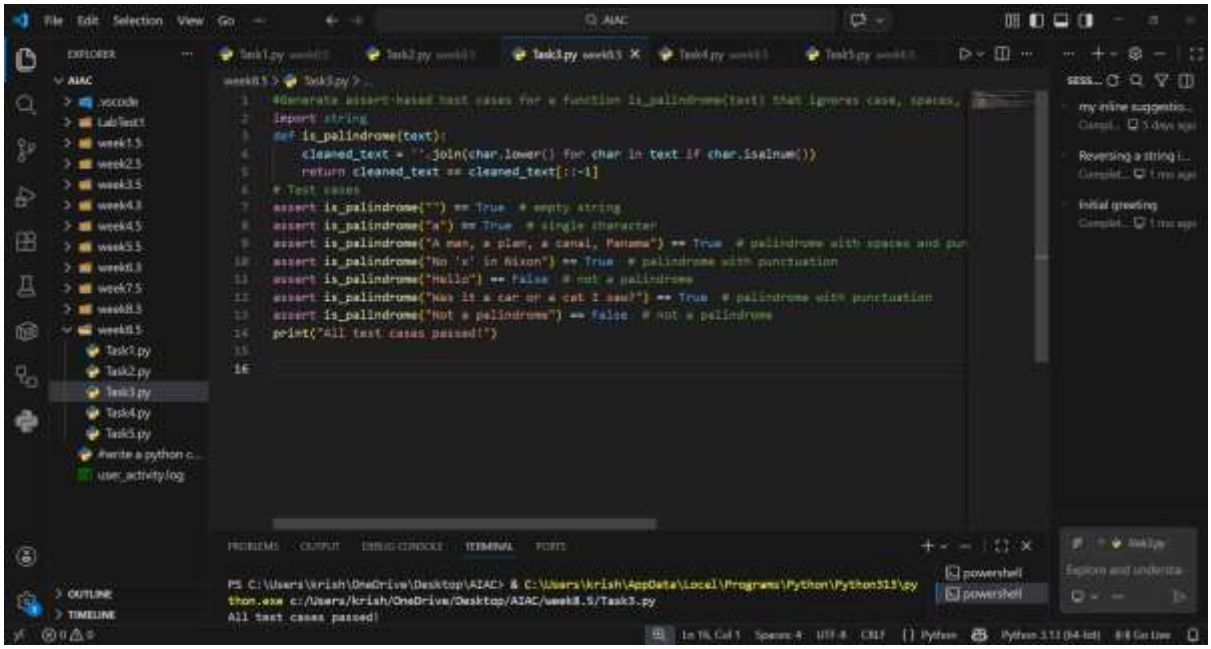
AI-generated tests.

PROMPT:

“Generate assert-based test cases for a function is_palindrome(text) that ignores case, spaces, and punctuation, and correctly handles empty strings and single characters.

import string”

CODE AND OUTPUT:



EXPLANATION:

Tests cover case-insensitivity, ignoring spaces/punctuation, empty strings, and single characters. The function normalizes input by removing non-alphanumeric and lowercasing. Then compares string with its reverse. TDD ensures correctness.

Task Description #4 (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development)

- **Task: Ask AI to generate at least 3 assert-based test cases for a BankAccount class and then implement the class.**
- **Methods:**

o deposit(amount)

o withdraw(amount)

o get_balance()

Example Assert Test Cases:

acc = BankAccount(1000)

acc.deposit(500)

assert acc.get_balance() == 1500

acc.withdraw(300)

assert acc.get_balance() == 1200

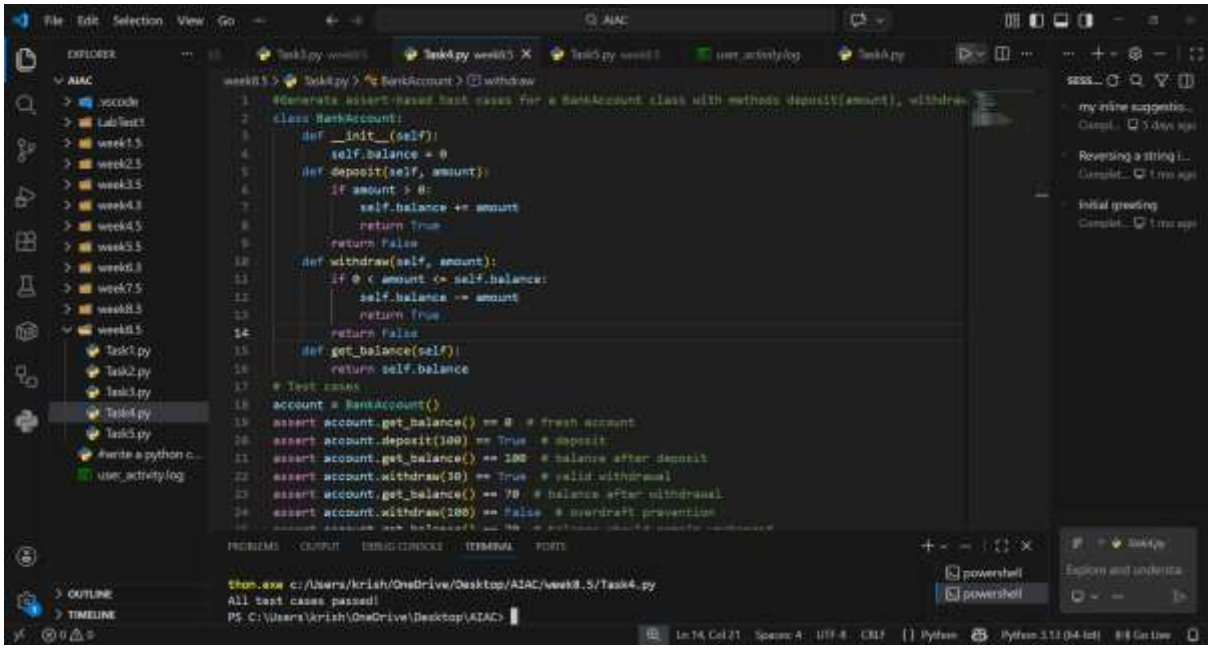
Expected Output #4:

• Fully functional class that passes all AI-generated assertions.

PROMPT:

"Generate assert-based test cases for a BankAccount class with methods deposit(amount), withdraw(amount), and get_balance(). Include cases for deposits, withdrawals, overdraft prevention, and fresh accounts."

CODE AND OUTPUT:



EXPLANATION:

Tests validate deposits, withdrawals, overdraft prevention, and multiple accounts. The class encapsulates balance, with methods to update and retrieve it. Withdraw checks available funds. TDD ensures all behaviors pass.

Task Description #5 (Email ID Validation – Apply AI for Data Validation)

• Task: Use AI to generate at least 3 assert test cases for a function validate_email(email) and implement the function.

• Requirements:

o Must contain @ and .

o Must not start or end with special characters.

o Should handle invalid formats gracefully.

Example Assert Test Cases:

assert validate_email("user@example.com") == True

assert validate_email("userexample.com") == False

assert validate_email("@gmail.com") == False

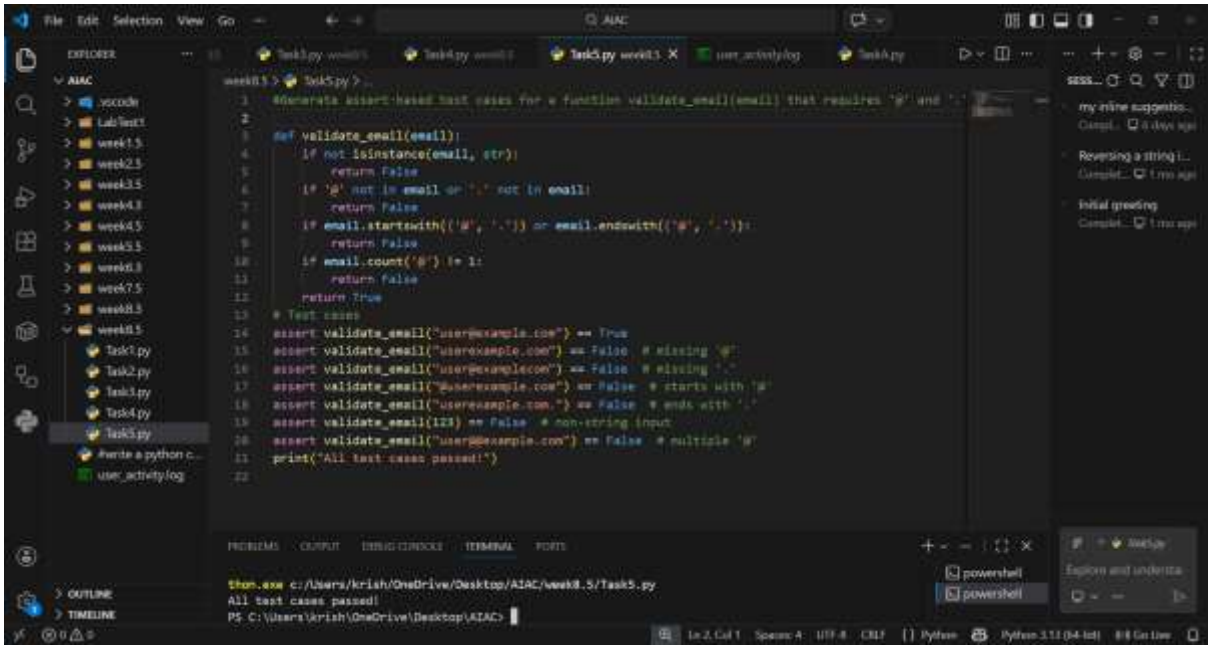
Expected Output #5:

• Email validation function passing all AI-generated test cases and handling edge cases correctly

PROMPT:

“Generate assert-based test cases for a function validate_email(email) that requires '@' and '.', disallows starting/ending with special characters, and handles invalid formats gracefully."

CODE AND OUTPUT:



EXPLANATION:

Tests include valid emails, missing symbols, invalid domains, and edge cases. The function checks presence of @ and ., ensures no leading/trailing special characters, and validates format. TDD ensures graceful handling.